

FastX

Online Bus Ticket Booking System

Technical Documentation

Version 1.0 | Milestone Project

1. Introduction

FastX is a full-stack online bus ticket booking system that connects passengers, bus operators, and platform administrators. Passengers can search routes across Indian cities, view seat availability, book tickets, and manage their bookings. Operators manage bus routes, monitor seat occupancy, and process refunds. Administrators oversee users, operators, bookings, and routes across the platform.

The application follows a modern three-tier architecture: an Angular 19 single-page frontend, a Flask REST API backend, and a MySQL database running in Docker. Authentication uses JWT tokens with role-based access control for user, operator, and admin roles.

2. Tech Stack

- Frontend: Angular 19 (standalone components, lazy-loaded routes, HttpClient)
- Backend: Python Flask 3.1 REST API with Flask-CORS
- Database: MySQL 8 (Docker container, schema auto-initialized)
- Authentication: PyJWT (HS256) with Bearer tokens
- Password hashing: Werkzeug generate_password_hash / check_password_hash
- Database driver: PyMySQL with DictCursor (requires cryptography for MySQL 8)
- Containerization: Docker Compose for MySQL
- Dev proxy: Angular dev server proxies /api to Flask on port 5000

3. Architecture

The system uses a client-server architecture. The Angular frontend runs on `http://localhost:4200` and communicates with the Flask API on port 5000 via REST endpoints prefixed with `/api`. During development, `proxy.conf.json` forwards API requests from the Angular dev server to the backend.

Browser (Angular SPA :4200)

|
| HTTP /api/*
v

Flask REST API (:5000)

|
| PyMySQL
v

MySQL 8 (Docker :3307 -> container :3306)

On startup, the Flask app calls `init_db()` which creates tables if missing, seeds default admin/operator accounts, and populates sample bus routes. JWT tokens are issued on login/register and stored in `localStorage` by the frontend.

4. Project Structure

Milestone-pro/

backend/

app.py	Flask REST API (all endpoints)
schema.sql	MySQL DDL and foreign keys
docker-compose.yml	MySQL 8 container config
requirements.txt	Python dependencies (includes cryptography)

frontend/

src/app/

pages/

login/ Login page

register/	User/operator registration
forgot/	Password reset
user/	Passenger dashboard
operator/	Operator dashboard
admin/	Admin dashboard
services/	
api.service.ts	HTTP client for all API calls
guards/	
auth.guard.ts	Route protection by role
app.routes.ts	Angular routing config
proxy.conf.json	Dev API proxy to :5000
package.json	Angular 19 dependencies
docs/	
generate_pdf.py	This documentation generator
FastX_Documentation.pdf	
README.md	Quick start guide
package.sh	Zip script (excludes docs/)

5. Installation Steps

Prerequisites: Python 3.10+, Node.js 18+, npm, Docker and Docker Compose.

Step 1 - Create a Python virtual environment and install backend dependencies:

```
cd backend
python3 -m venv venv
source venv/bin/activate    # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

Step 2 - Start MySQL and the Flask API:

```
cd backend
source venv/bin/activate
docker compose up -d
python3 app.py
```

Step 3 - Start the frontend (new terminal):

```
cd frontend
npm install
npm start
```

Step 4 - Open <http://localhost:4200> in your browser.

MySQL is exposed on host port 3307 (mapped to 3306 inside the container) to avoid conflicts with a local MySQL installation. Default DB password is root; no extra environment variables are required for local development.

Optional environment variables: DB_HOST (localhost), DB_PORT (3307), DB_USER (root), DB_PASS (root), DB_NAME (fastx), JWT_SECRET (fastx-secret-key).

6. Default Logins

Role	Email	Password
Admin	admin@fastx.com	admin123
Operator	operator@fastx.com	operator123

Passengers must register at /register. Login accepts email or username. After login, users are redirected to /user, /operator, or /admin based on role.

7. User / Operator / Admin Features

Passenger (User) Features

- Search buses by origin, destination, and travel date with city autocomplete
- View route details: bus type, fare, amenities, departure/arrival times
- Interactive seat map with real-time availability
- Book one or more seats; total fare calculated automatically
- Payment step with Card or UPI (client-side validation; simulated checkout)
- E-ticket view with booking details after successful payment
- View and cancel confirmed bookings
- View and update profile (name, gender, phone, address)
- Download/print ticket for confirmed bookings

Operator Features

- Add, edit, and delete bus routes (name, number, type, seats, times, fare)
- Configure amenities: water bottle, blanket, charging point, TV
- Check seat availability by route and date
- View all bookings on operator routes with passenger details
- Process refunds for cancelled bookings
- Update operator profile

Admin Features

- View and delete passenger accounts
- View and delete operator accounts
- View all bookings system-wide with status badges
- View and delete any bus route across all operators

8. How to Use Guide

For Passengers

- Register at /register or login at /login
- On the Find Bus tab, pick origin and destination from the city dropdown, enter date, and click Search Buses
- Select a bus from results; choose seats on the seat map; proceed to payment
- Complete payment with Card or UPI details; view the e-ticket on confirmation
- View tickets under My Bookings; cancel if needed before travel
- Update personal details under Profile

For Operators

- Login with operator@fastx.com / operator123
- Routes tab: add new routes or edit/delete existing ones
- Seat Availability tab: pick route and date to see booked vs available seats
- Bookings tab: review passenger bookings; process refunds for cancelled ones

For Admins

- Login with admin@fastx.com / admin123
- Use tabs to manage Users, Operators, Bookings, and Routes
- Delete records as needed to maintain the platform

9. REST API Reference

Base URL: <http://localhost:5000/api> | Auth: Authorization: Bearer <token>

Method	Endpoint	Role	Description
POST	/api/register	Public	Register user/operator
POST	/api/login	Public	Login, returns JWT token
POST	/api/forgot-password	Public	Reset password by email
GET	/api/locations?q=	Public	City autocomplete list
GET	/api/profile	Auth	Get current user profile
PUT	/api/profile	Auth	Update profile
GET	/api/routes/search	Public	Search routes (origin, destination, date)
GET	/api/routes/<id>	Public	Route details
GET	/api/routes/<id>/seats?date=	Public	Seat availability
POST	/api/bookings	User	Create booking
GET	/api/bookings	User	List my bookings
GET	/api/bookings/<id>	User	Booking detail / ticket
DELETE	/api/bookings/<id>	User	Cancel booking
GET/POST	/api/operator/routes	Operator	List / add routes
PUT/DELETE	/api/operator/routes/<id>	Operator	Update / delete route
GET	/api/operator/bookings	Operator	Operator bookings
GET	/api/operator/routes/<id>/seats	Operator	Seat map for route
POST	/api/operator/refund/<id>	Operator	Process refund
GET	/api/admin/users	Admin	List passengers
DELETE	/api/admin/users/<id>	Admin	Delete passenger
GET	/api/admin/operators	Admin	List operators
DELETE	/api/admin/operators/<id>	Admin	Delete operator
GET	/api/admin/bookings	Admin	All bookings
DELETE	/api/admin/bookings/<id>	Admin	Delete booking
GET	/api/admin/routes	Admin	All routes
DELETE	/api/admin/routes/<id>	Admin	Delete route

10. Database Schema

Database name: fastx

users

- id INT PK AUTO_INCREMENT
- name VARCHAR(100), email VARCHAR(120) UNIQUE, password VARCHAR(255)
- gender VARCHAR(20), phone VARCHAR(20), address TEXT
- role ENUM('user','operator','admin') DEFAULT 'user'
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

routes

- id INT PK, operator_id INT FK -> users(id) ON DELETE CASCADE
- bus_name, bus_number, origin, destination VARCHAR
- bus_type ENUM('sleeper_ac','sleeper_non_ac','seat_ac','seat_non_ac')
- total_seats INT, departure_time TIME, arrival_time TIME

- fare DECIMAL(10,2), amenities JSON

bookings

- id INT PK, user_id INT FK -> users(id), route_id INT FK -> routes(id)
- travel_date DATE, seats JSON, total_fare DECIMAL(10,2)
- status ENUM('confirmed','cancelled','refunded') DEFAULT 'confirmed'
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

11. Security

- Passwords hashed with Werkzeug before storage; never returned in API responses
- JWT tokens expire after 24 hours (HS256 signed with JWT_SECRET)
- Role-based access via @auth_required decorator on protected endpoints
- Operators can only modify their own routes and bookings
- Users can only view/cancel their own bookings
- Admin endpoints restricted to role='admin'
- Input validation on registration, profile updates, and booking creation
- CORS enabled for frontend cross-origin requests during development
- Note: Change JWT_SECRET and DB credentials in production; debug mode is enabled in app.py for development only.

12. Available Routes (Seed Data)

On first startup, 48 sample routes are seeded: 24 outbound legs plus 24 return legs (3 buses per direction for each city pair). Supported cities: Mumbai, Delhi, Bangalore, Chennai, Hyderabad, Kolkata, Pune, Ahmedabad, Jaipur, Lucknow, Kochi, Goa, Surat, Nagpur, Indore, Bhopal, Patna, Chandigarh. Example search: Mumbai to Pune shows 3 buses.

- Mumbai -> Pune (Volvo Express, Neeta Travels, Purple Line)
- Mumbai -> Bangalore (Shivneri, VRL Travels, SRS Express)
- Mumbai -> Delhi (Rajdhani Deluxe, National Express, Maharashtra Travels)
- Chennai -> Hyderabad (Orange Travels, KPN Travels, Jabbar Express)
- Delhi -> Jaipur (Royal Cruiser, Pink City Express, Rajasthan Roadways)
- Bangalore -> Chennai (KPN Travels, Orange Tours, Green Express)
- Pune -> Goa (Paulo Travels, Konkan Kanya, Goa Express)
- Bangalore -> Kochi (KSRTC Swift, Kallada Travels, Kerala Express)

Return routes are auto-generated for each outbound route.

13. Troubleshooting

- Cannot connect to database: Ensure Docker is running and 'docker compose up -d' succeeded. The app connects on port 3307 with password root by default.
- pip install fails on Linux: Use a virtual environment (python3 -m venv venv) instead of system pip to avoid externally-managed-environment errors.
- MySQL authentication error: Install the cryptography package (listed in requirements.txt) for PyMySQL with MySQL 8.
- API calls fail from frontend: Verify Flask is running on port 5000 and Angular dev server uses proxy.conf.json (npm start includes proxy by default).
- Login returns 401: Check credentials. Default admin/operator accounts are seeded on first app startup via init_db().
- Session expired: JWT tokens last 24 hours. Log in again to get a fresh token.
- No buses found: Use seeded city pairs listed in Section 12. Date must be today or later.
- Seat booking fails: Seat may already be taken. Refresh seat map and pick available seats.
- Port 3306 already in use: The project defaults to host port 3307. If that port is busy, change the mapping in

docker-compose.yml and set DB_PORT accordingly.

- TypeScript config warning (outDir): Set rootDir to ./src in frontend/tsconfig.json when using TypeScript 6 or newer.
- Regenerate this PDF: python3 docs/generate_pdf.py